NYCU Introduction to Machine Learning, Homework 4

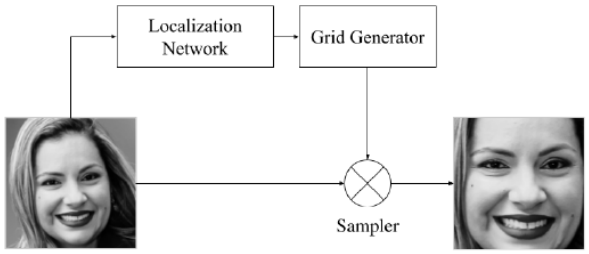
[112550069], [謝侑哲]

Part. 1, Kaggle (70% [50% comes from the competition]):

(10%) Implementation Details

I use an open source model – EmoNeXt to achieve this task. There are 5 main components in EmoNeXt and then compose the whole model. I will talk component by component.

The first component is Spatial Transformer Networks. This component is used to cut the image to only face part making the model will not be influenced by noise like hair etc. Below is its architecture:



The Localization Network is a CNN and it will extract position and semantic information of the image. Then, the extracted feature will be thrown to Grid Generator (a fully-connected layer). It will generate a grid and there will be position information in each cell. Then, Sampler will identity map the origin image to output according to the grid cells' value.

The second component is ConvNeXt. It's a mature CNN architecture by improving ResNet. Because it's famous and just did some simple tricks (eg: modify dim, use MobileNet's technique) I will not introduce much more about this network.

The third component is Squeeze-and-Excitation. Its architecture is like below:

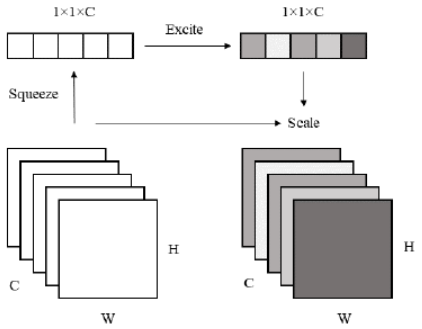


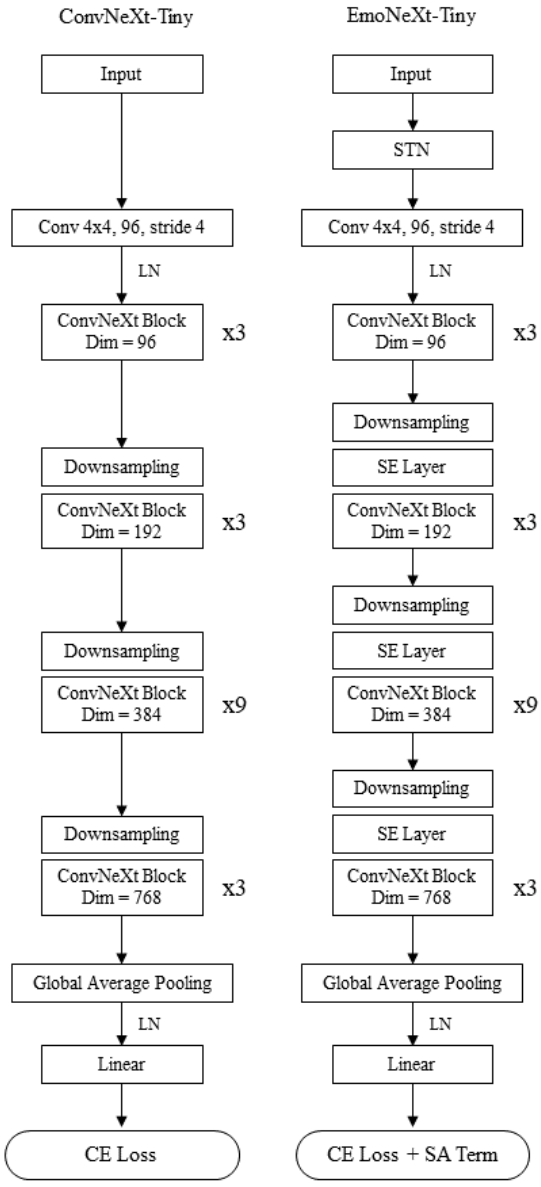
Fig. 3. The architecture of the Squeeze-and-Excitation block.

There two operations, Squeeze and Excite (SE Layer). In Squeeze, it uses global average pooling to average the value of a whole feature map. Then Excite is a fully-connected layer, and it will model the correlation in each feature map by Squeeze's outcome. Finally, generate a weight to apply on the feature map. It can be imagined as a simple attention mechanism but

uses less computational cost. It will make the feature map which is more representative be valued by the model.

The fourth component is simply a Self-Attention (SA) term in loss, it is to compute the importance (weight) of different features to avoid the model focus on a little feature and ignore a part of features.

Below is the whole architecture of EmoNeXt, which is base on ConvNeXt:



Besides directly using this model, I use other training strategy:
1. Because of the unbalanced data classes in training data, I apply different weights to different classes.
2. Because data is not so enough I implement data augmentation.
3. To enhance robustness, I train 5 model and let them vote for the final result.

In addition, I also reconstructed a dataset and tester to ensure the model was running well.

Model backbone (e.g., VGG16, VGG19, Custom, etc)	EmoNeXt (https://ieeexplore.ieee.org/abstract/document/10337732)
Number of model parameters	91765259 (I use base version model)
Other hyperparameters …	lr=0.0001, loss weight: [1.0, 1.5, 1.0, 1.0, 1.0, 1.0, 0.5], weight_decay=1e-3

Model Pipeline:
Train 5 model wight -> use tester to inference 5 csv (output{i}.csv) -> use vote_csv.py to get vote result

Training Command:

```
python train.py \
    --dataset-path='data/Images' \
    --batch-size=64 --lr=0.0001 \
    --epochs=300 \
    --amp \
    --in_22k \
    --num-workers=4 \
    --model-size='base' \
    --output-dir 'out' \
    --device
```

Testing Command:

```
python tester.py \
    --dataset-path='data/Images' \
    --amp \
    --in_22k \
    --num-workers=1 \
    --model-size='base'
```

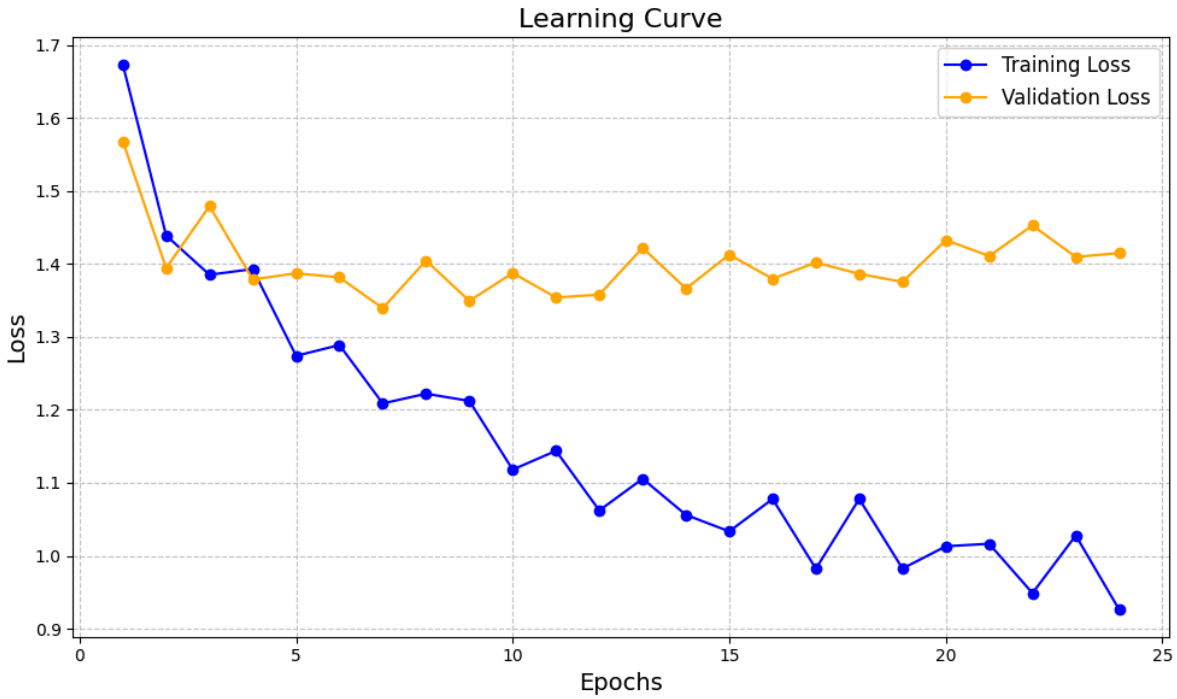
Vote Commad:

```
python csv_vote.py
```

(10%) Experimental Results

Evaluation metrics: loss(cross_entropy), accuracy

Training Curve:



Ablation Study 1: different weight decay
(use random split, train:validation = 95:5, lr=1e-4, choose best validation result):

Regualrization Term (Weight Decay)	1e-2	1e-3	1e-4	1e-5
Training Loss	0.9368	1.0851	1.0561	1.0312
Training Accuracy	91.7960%	83.7968%	84.5246%	86.8922%
Validation Loss	1.3992	1.3233	1.3662	1.3648
Validation Accuracy	71.3788%	71.7967%	71.0306%	71.3788%

We can observe 1e-3 is the best weight decay value to ensure both the model's capacity and generalizability.

Ablation Study 2: with or without SA term
(lr=1e-4, weight decay=1e-3)
(use random split, train:validation = 95:5, choose best validation result):

	w/ SA term	w/o SA term
Training Accuracy	83.7968%	93.5739%
Validation Accuracy	71.7967%	72.4930%

I'm not really sure why the model without the SA term will perform well. In my other experiments, models without the SA term are usually better. However, I think it is because SA terms force the model to consider all features rather than only focus on some features. Nonetheless, in this task, the images may be not so complex and making a model only focusing on some features is acceptable because some features are enough to represent human's emotion.

Part. 2, Questions (30%):

1. (10%) Explain the support vector in SVM and the slack variable in Soft-margin SVM. Please provide a precise and concise answer. (each in two sentences)

   SVM tries to find the maximum margin to classify classes. The support vector is the vector (feature) deciding the margin, which is precisely on the margin.

   SVM needs to tolerate some classification error because not all data is speartable. The slack variable is to model the misclassification level, which is a variable in constraint of how far a data point should be to the hyperplane (t * y(x) >= 1 - slack variable)

2. (10%) In training an SVM, how do the parameter C and the hyperparameters of the kernel function (e.g., γ for the RBF kernel) affect the model's performance? Please explain their roles and describe how to choose these parameters to achieve good performance.

   These hyperparameters decide the level that the model can tolerate misclassification. In general, they mean the model tends to overfit (tolerate too less misclassification) or underfit (tolerate too much misclassification).

   Parameter C is the coefficient of slack variable term. If C is bigger, the model will tolerate less misclassification because misclassification's loss is bigger. On the other hand, smaller C will make the model be able to tolerate more misclassification.

   In the case of the RBF kernel, gamma is the reciprocal of variance of the kernel, which restricts the range of kernel function(as normalization coefficient). If the gamma is bigger, the kernel function will be in a bigger range. It will lead to the model having more information to adapt, which may cause overfitting. If the gamma

is smaller, the kernel function will be in a smaller range. It restricts the change of data after mapping, which may cause underfitting.

If We want to choose good hyperparameters to achieve good parameters. We can observe the overfitting or underfitting situation by training loss and validation loss first. If training loss is much less than validation loss after enough training, it overfits. We can choose smaller C and gamma. If training loss and validation is close and both are high after enough training, it underfit. We can choose bigger C and gamma.

3. (10%) SVM is often more accurate than Logistic Regression. Please compare SVM and Logistic Regression in handling outliers.

SVM uses soft margin to handle outliers. Like what I mentioned in problem 1, it can tolerate different level outliers by adjusting hyper parameters C. Thus, some outliers will not influence SVM to construct a good enough hyperplane to classify.

However, logistic regression lacks a mechanism to handle outliers. Logistic Regression try to model the posterior distribution P(Ci|X). If an outlier has C1's most property and C2's label, it will directly influence the maximum log likelihood's result. Besides, logistic regression uses cross entropy error which will lead to large loss when a probability is wrong (log(y) is large when y is close to 0). Therefore, logistic regression will be impacted by outliers a lot.